

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:100 Mode: In Class Hours : 60 mts

Annexure A

EXPERIMENTS

*I M Harinath/192425020 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

EXPERIMENTS

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax directed definitions (SDD). The system must:

(i) Accept arithmetic expressions with operators +, -, *, / and parentheses (ii)

Construct a syntax tree and evaluate expressions using synthesized attributes

(iii) Respect operator precedence and associativity rules

(iv) Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Aim: To evaluate arithmetic expressions using syntax-directed definitions with synthesized attributes.

To construct a syntax tree and compute the final value by applying operator precedence and associativity rules.

Algorithm:

1. Read two type expressions.
2. Compare both type strings.
3. If both are identical, they are equivalent.
4. Otherwise, they are not equivalent.
5. Display the result.

C Program:

```
#include <stdio.h>
```

```
int evaluate(int a, int b, char op) {
```

```
    switch(op) {
```

```
        case '+': return a + b;
```

```
        case '-': return a - b;
```

```
        case '*': return a * b;
```

```
        case '/': return a / b;
```

```
    default: return 0;
```

```

    }
}

int main() {

    int a, b, c;

    int temp, result;

    printf("Syntax Directed Evaluation using Synthesized Attributes\n");

    printf("Enter values for a, b and c: ");

    scanf("%d %d %d", &a, &b, &c);

    temp = evaluate(b, c, '*');

    result = evaluate(a, temp, '+');

    printf("Expression: a + b * c\n");

    printf("Intermediate Value (b * c) = %d\n", temp);

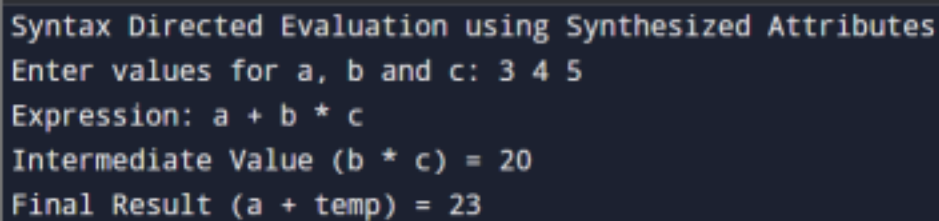
    printf("Final Result (a + temp) = %d\n", result);

    return 0;

}

```

Output:



```

Syntax Directed Evaluation using Synthesized Attributes
Enter values for a, b and c: 3 4 5
Expression: a + b * c
Intermediate Value (b * c) = 20
Final Result (a + temp) = 23

```

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression
- (iii) Demonstrate bottom-up computation of intermediate results
- (iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S

attributed definitions in compilers.

Aim: To evaluate a postfix expression using a stack-based method in C. To demonstrate bottom-up evaluation and the use of S-attributed definitions in compiler design.

Algorithm:

1. Read the postfix expression.
2. Scan each character.
3. Push operands onto the stack.
4. Pop two operands when an operator is encountered.
5. Perform the operation and push the result.
6. Repeat until the expression ends.
7. Display the final result.

Code:

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int stack[100];

int top = -1;

void push(int value) {
    stack[++top] = value;
}

int pop() {
    return stack[top--];
}

int main() {
    char postfix[100];

    int i, a, b, result;

    printf("Postfix Expression Evaluation\n");

    printf("Enter postfix expression: ");
```

```

scanf("%s", postfix);

for(i = 0; i < strlen(postfix); i++) {
if(isdigit(postfix[i])) {

push(postfix[i] - '0');

} else {

b = pop();

a = pop();

switch(postfix[i]) {

case '+': result = a + b; break; case '-':
result = a - b; break; case '*': result = a
* b; break;

case '/': result = a / b; break; default:
result = 0;

}

push(result);

}

}

printf("Final Result = %d\n", pop());

return 0;

}

```

Output:

```
Postfix Expression Evaluation
Enter postfix expression: 24*+5-2
Final Result = 2

=== Code Execution Successful ===
```

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Aim:

To simulate L-attributed definition for arithmetic expression evaluation using inherited attributes.

To perform expression evaluation in a top-down parsing approach while maintaining operator precedence.

Algorithm:

1. Read two operands and an operator.
2. Pass operand values as inherited information.
3. Perform the operation.
4. Display the result.

C Program:

```
#include <stdio.h>

int evaluate(int a, int b, char op) {

    switch(op) {

        case '+': return a + b;

        case '-': return a - b;
```

```

case '*': return a * b;

case '/': return a / b;

}

return 0;

}

int main() {

int a, b;

char op;

printf("Enter expression (a op b): ");

scanf("%d %c %d", &a, &op, &b);

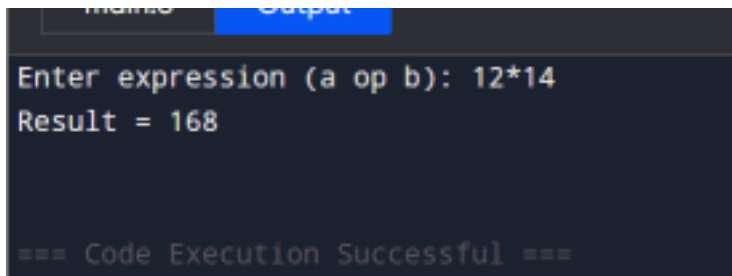
printf("Result = %d\n", evaluate(a, b, op));

return 0;

}

```

Output:



```

Enter expression (a op b): 12*14
Result = 168

=== Code Execution Successful ===

```

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int*, char*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Aim:

To check the equivalence of two data types including basic and pointer types.

To simulate type equivalence checking performed during the semantic analysis phase of a compiler.

Algorithm:

1. Read two type expressions.
2. Compare both type strings.
3. If both are identical, they are equivalent.
4. Otherwise, they are not equivalent.
5. Display the result.

C Program

```
#include <stdio.h>

#include <string.h>

int main() {

    char t1[20], t2[20];

    printf("Enter first type: ");

    scanf("%s", t1);

    printf("Enter second type: ");

    scanf("%s", t2);

    if(strcmp(t1, t2) == 0)

        printf("Types are Equivalent\n");

    else

        printf("Types are Not Equivalent\n");

    return 0;

}
```

Output


```
Enter first type: int
Enter second type: int
Types are Equivalent

=== Code Execution Successful ===
```

Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types
- (ii) Validate operations such as +, -, , /
- (iii) *Detect invalid combinations such as int + char*
- (iv) Report whether the expression is valid or contains a type error

Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

Aim:

To perform type checking for arithmetic expressions using operand types and operators.
To detect type incompatibility errors and ensure semantic correctness during compilation.

Algorithm:

1. Read the types of two operands.
2. Read the arithmetic operator.
3. Check whether both operands are compatible.
4. If compatible, the expression is valid.
5. Otherwise, report a type error.

C Program

```
#include <stdio.h>

#include <string.h>

int main() {

    char t1[20], t2[20], op;
    printf("Enter first type: ");
```

```

scanf("%s", t1);

printf("Enter operator (+,-,*,/): ");

scanf(" %c", &op);

printf("Enter second type: ");

scanf("%s", t2);

if((strcmp(t1, "int") == 0 || strcmp(t1, "float") == 0) &&
(strcmp(t2, "int") == 0 || strcmp(t2, "float") == 0))

printf("Expression is Valid\n");

else

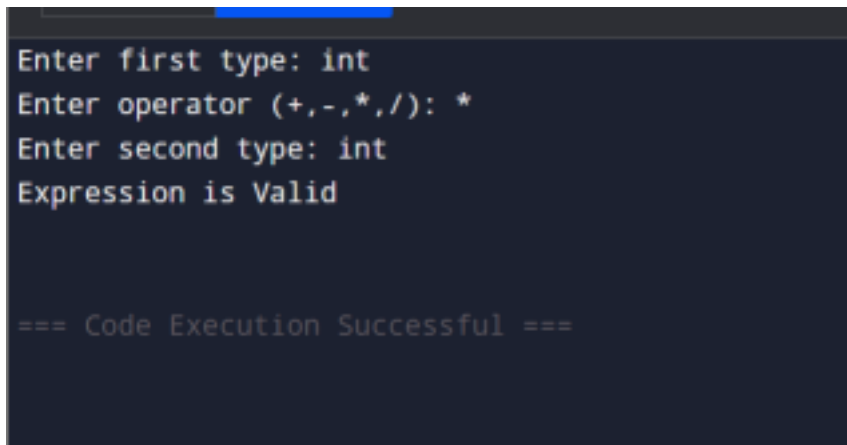
printf("Type Error: Invalid Expression\n");

return 0;

}

```

Output:



```

Enter first type: int
Enter operator (+,-,*,/): *
Enter second type: int
Expression is Valid

=== Code Execution Successful ===

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts

		concepts/technologies			
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation